

Digital Image Processing Lab



Name: Ipshita Gupta

Section: O

Class Roll No.: 29

University Roll No: 2315000989

Subject Code: BCSE 0131

Subject Title: DIGITAL IMAGE PROCESSING

Submitted to :
Ajay Kumar Mahato
Assistant Professor, ECE Department,
GLA University, Mathura

Exp. Number	Aim of the Experiment	Date of Performance	Page Number	Remarks
1.	Perform the various Matrix operations.	9 jan 2026		
2.	Perform conversion of image	9 jan 2026		
3.	Perform image resize	16 jan 2026		
4.	Extraction of Colour Band	23 jan 2026		
5.	Perform different Image crop.	30 jan 2026		
6.	Perform different Image crop.	6 feb 2026		
7.	Write a program to plot different function like sy function, exponential wave and pdt of exponential and side by using stem command	13 feb 2026		
8.	Perform various Image Enhancement operations: Image Negation function, Logarithmic Transformation, Power Law Transformation	20 mar 2026		
9.	Perform Histogram Equalization, contrast stretching	13 mar 2026		
10.	Sharpen an image using Laplacian filter.	10 apr 2026		
11.	Perform various Fast Fourier transform (FFT) and frequency domain filtering on images using MATLAB.	24 apr 2026		

Contents

Experiment no: 1

AIM OF THE EXPERIMENT: Perform the various Matrix operations.

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

In Digital Image Processing (DIP), images are represented in the form of matrices, where each element corresponds to a pixel intensity value. Since images are numerical matrices, most of the operations performed on images are based on matrix manipulation and arithmetic operations.

MATLAB (Matrix Laboratory) is a powerful computational software used extensively for performing mathematical, engineering, and image processing tasks. It treats almost all data as matrices, making it ideal for performing image-based calculations and transformations efficiently.

This experiment focuses on performing various matrix and arithmetic operations using MATLAB to understand how data is stored, accessed, modified, and processed in the form of matrices.

1. Arithmetic Operations

Arithmetic operations are performed on numerical or image data to manipulate pixel intensity or perform mathematical calculations. In MATLAB, these operations are performed directly on variables or arrays.

Common arithmetic operations include:

- **Addition (+):**
Adds two scalars, vectors, or matrices of the same dimension.
Example: $c = a + b$
➤ Used to increase brightness in images or combine pixel intensities.
- **Subtraction (-):**
Subtracts one matrix or scalar from another.
Example: $d = a - b$
➤ Used to find differences between pixel values or perform change detection.
- **Multiplication (*) and Division (/):**
Performs element-wise or matrix multiplication/division depending on the operator used.
Example:
 - Scalar multiplication: $e = a * b$
 - Division: $f = a / b$
➤ Used for contrast scaling or intensity normalization in image processing.

MATLAB also provides **element-wise operators** (**.***, **./**, **.^**) to perform arithmetic on corresponding elements of matrices.

2. Matrix Creation and Indexing

Matrices are created using square brackets []. MATLAB supports different types of matrix initialization:

- **Row and Column Vectors**

Example:

- Row vector $\rightarrow G = [1\ 2\ 3\ 4\ 5\ 6\ 7]$
- Column vector $\rightarrow J = [1;2;3;4;5;6]$

- **Sequence Generation**

Using the colon operator (:):

$H = 1:7$ generates numbers from 1 to 7.

$I = 1:2:7$ generates numbers from 1 to 7 with a step size of 2.

- **Matrix Indexing**

MATLAB allows element access using indices:

- $N(1,2) \rightarrow$ element at Row 1, Column 2
- $N(2,:) \rightarrow$ all elements of Row 2
- $N(:,3) \rightarrow$ all elements of Column 3
- $N(1,2:3) \rightarrow$ elements from Column 2 to 3 of Row 1

3. Matrix Concatenation

Concatenation refers to **joining matrices** either **horizontally** or **vertically**.

- **Horizontal Concatenation:** $[A\ B]$ joins matrices side by side.
- **Vertical Concatenation:** $[A; B]$ joins matrices one below the other.

Example:

$V = [N\ U]$ % horizontal concatenation

$W = [N; U]$ % vertical concatenation

This is useful for image stitching, merging, or building larger matrices from smaller ones.

4. Random Matrix Generation

MATLAB provides several built-in functions to generate matrices filled with random numbers:

- `rand(m,n)` → Generates an $m \times n$ matrix with uniformly distributed random numbers between 0 and 1.
- `randn(m,n)` → Generates an $m \times n$ matrix with normally distributed (including negative) random values.
- `randi([min max], m, n)` → Generates an $m \times n$ matrix with random integer values within a specified range.

These are useful in simulating noise or generating test data in image processing experiments.

5. Sorting Operations

Sorting arranges data in ascending or descending order:

- `sort(X)` → Sorts each column in ascending order.
- `sort(X, 'descend')` → Sorts each column in descending order.
- `sortrows(X)` → Sorts the matrix row-wise.

Sorting helps in applications like histogram equalization, ranking, or ordering pixel intensity values.

6. Special Matrix Generation

MATLAB provides several functions to create predefined matrices useful in numerical and image applications:

- `zeros(m,n)` → Creates an $m \times n$ matrix filled with zeros.
- `ones(m,n)` → Creates an $m \times n$ matrix filled with ones.
- `eye(n)` → Creates an $n \times n$ identity matrix.
- `triu(A)` → Extracts the upper triangular part of a matrix.
- `tril(A)` → Extracts the lower triangular part of a matrix.

These matrices are commonly used in image transformations, kernel design, and matrix decomposition.

7. Matrix Reshaping and Rotation

- **Reshaping (reshape):**
Rearranges elements of a matrix into a new shape without changing data.
Example: `reshape(A, 2, 6)` converts the matrix into 2 rows and 6 columns.

- **Rotation (rot90):**
Rotates a matrix by 90 degrees:
 - `rot90(A)` → Rotates anticlockwise.
 - `rot90(A, -1)` → Rotates clockwise.

These operations are important in geometric transformations and image rotation processes.

8. Matrix Multiplication and Element-wise Operations

- **Matrix Multiplication (*):**
Follows the standard rule of matrix multiplication, where the number of columns of the first matrix equals the number of rows of the second.
Example: `ao = am * an`
- **Element-wise Multiplication (.*):**
Multiplies corresponding elements of two matrices of the same size.
Example: `ap = am .* an`

These are fundamental in image masking, blending, and filtering operations.

9. Variable Management Commands

- `who` → Displays the names of all variables in the workspace.
- `whos` → Displays detailed information (size, bytes, class) about variables in the workspace.

These commands are essential for debugging and understanding the workspace environment.

CODE:

```
clc ;           %clear the command window
clear all;      %clear the workspace (variable)
close all;      %close command window

a = 10 ;
b = 20;

%Arithmetic Operations

c = a+b
d = a-b
```

`e = a*b`

`f = a/b`

`who` % command to see all the variables

`whos` %Details about variables

`G=[1 2 3 4 5 6 7]` %row vector

`H=1:7`

`I=[1:2:7]` % [starting : updation : ending] works without square bracket as well

`J = [1;2;3;4;5;6]` %column vector

`K= J(4)` %element/Indexing Extraction

`I=J(4:6)` % extract from index 4 to 6

`N = [1 2 3 ; 4 5 6 ; 7 8 9]` %gives the matrix from 1 to 9

`o=[1:3;4:6;7:9]`

`p = N(1,2)` %gives the element of matrix (N) as N(Row , Column)

`Q = N(1,2:3)` %gives the element from row 1 and column 2 to column 3

`R = N(1,[2 3])` %this give the same

`S = N(2,:)` %give all the element of Row 1

`T = N(2,1:3)` % this is also the same

%Concatenation

`U=[3 4 5 ; 6 7 8 ;9 10 11]`

`V=[N U]` %assembles the columns of the matrix

`W=[N;U]` %assembles in rows

% Random Data Generation

`X=rand(3)` % Generates the 3x3 matrix with random values

`Y=randn(3)` %Generates the 3x3 matrix with some negative value

`Z=randi([1 10],3,4)` % Generates the 3x3 matrix with integer value

%Sorting

`aa = sort(X)` %sort column wise in ascending order

`ab = sortrows(X)` %sorts rowise

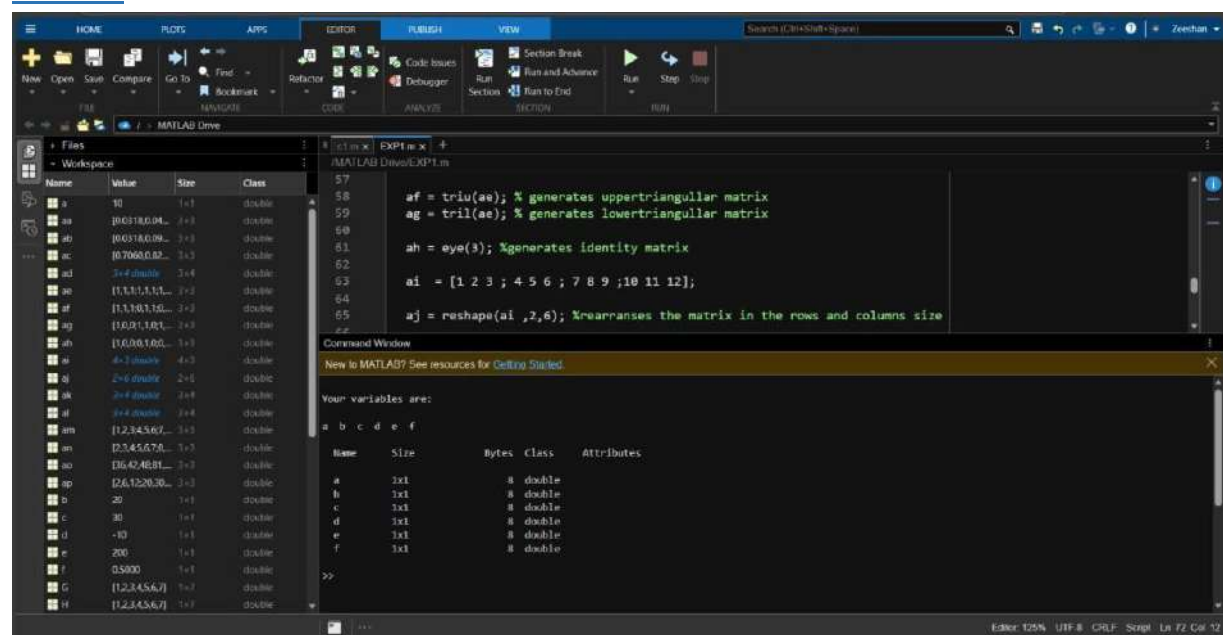
`ac = sort(X,'descend')` %sorts column wise in descending order

```

ad = zeros(3,4) % Generates 3x4 Zero matrix
ae = ones(3,3) %Generates One matrix of 3x3
af = triu(ae) % generates uppertriangular matrix
ag = tril(ae) % generates lowertriangular matrix
ah = eye(3) %generates identity matrix
ai = [1 2 3 ; 4 5 6 ; 7 8 9 ;10 11 12]
aj = reshape(ai ,2,6) %rearranges the matrix in the rows and columns size
ak = rot90(ai) %rotates the matrix anti-clockwise
al = rot90(ai,-1) %rotates the matrix clockwise
am = [1 2 3 ; 4 5 6 ; 7 8 9]
an = [2 3 4 ; 5 6 7 ;8 9 10]
ao = am*an %multiply the matrix
ap =am.*an %element wise multiplication we can use ./ as well

```

RESULT:



CONCLUSION:

Matrix and arithmetic operations form the foundation of MATLAB's capabilities in digital image processing. By mastering these operations, we can effectively perform image enhancement, transformation, filtering, and analysis. Understanding how MATLAB handles matrices provide the base for advanced DIP operations like convolution, correlation, and morphological processing.

Experiment no: 2

AIM OF THE EXPERIMENT: Perform different Image type conversion.

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

1. Color Image

A color image is composed of three channels — Red, Green, and Blue (RGB). Each pixel contains intensity values for all three colors. These combined values form a full-color image. In MATLAB, the image is displayed using all three color planes together.

2. Grayscale Image Conversion

A grayscale image is obtained from the color image by eliminating hue and saturation while retaining only brightness information. Each pixel represents an intensity level from 0 (black) to 255 (white). This simplifies image data and reduces it to a single channel, which is easier to process for operations like edge detection and segmentation.

3. Binary Image Conversion using im2bw()

In this conversion, the grayscale image is converted into a binary image using a threshold value. Pixels with intensity higher than the threshold are converted to white (1), and others to black (0). This process is called thresholding and helps in separating objects from the background.

4. Binary Image Conversion using imbinarize()

imbinarize() is a more advanced function for converting grayscale images into binary images. It allows both manual and automatic thresholding, providing better accuracy and flexibility. The result is a two-level (black and white) image suitable for further morphological operations.

5. Binary Image Conversion without Inbuilt Function

In this method, binary conversion is performed manually by applying a logical condition on pixel values. If a pixel's intensity is greater than a chosen threshold (e.g., 128), it is assigned as white (1), otherwise black (0). This approach helps in understanding the concept of thresholding without using built-in MATLAB functions.

Code:

```
clc;  
  
clear all;
```

```
close all;
```

```
a=imread("Lenna_(test_image).png")
```

```
subplot(3,2,1)
```

```
imshow(a)
```

```
title('color image')
```

```
b = rgb2gray(a)
```

```
subplot(3,2,2)
```

```
imshow(b)
```

```
title('gray image')
```

```
c = im2bw(b,0.5);
```

```
subplot(3,2,3);
```

```
imshow(c);
```

```
title('binary image with im2btw')
```

```
d=imbinarize(b,0.5);
```

```
subplot(3,2,4);
```

```
imshow(c);
```

```
title('binary image with binarize')
```

```
e = b>128;
```

```
subplot(3,2,5)
```

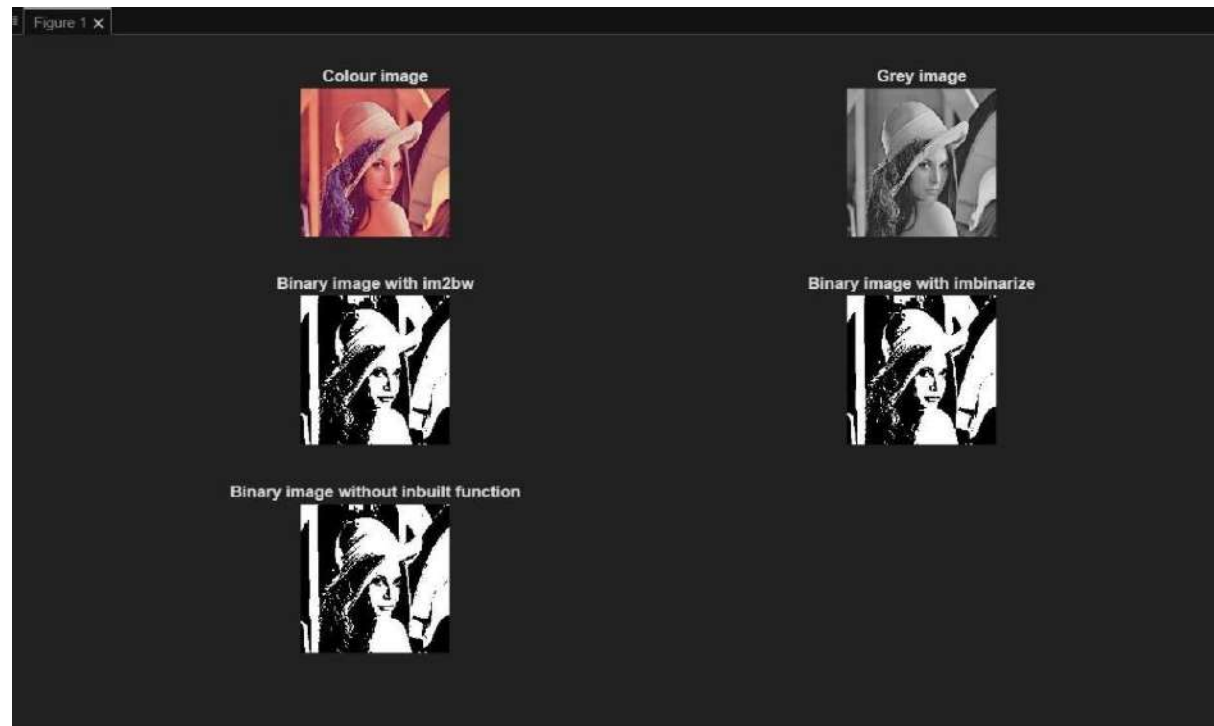
```
imshow(e);
```

```
title('binary without inbuilt function');
```

Conclusion:

In this experiment, various image conversions were successfully performed using MATLAB. A color image was first converted into a grayscale image by retaining only intensity information and removing color components. The grayscale image was then converted into binary form using different thresholding techniques — including built-in functions (im2bw, imbinarize) and manual logical operations.

Result:



Experiment no: 3

AIM OF THE EXPERIMENT: Perform different Image type of colour band

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

A color image in MATLAB is stored as a **3-dimensional matrix** of size $M \times N \times 3$, where:

- **M** → number of rows (height of image)
- **N** → number of columns (width of image)
- **3** → the three color planes (R, G, B)

In this experiment, the process of **color band extraction** involves separating each of these three channels from the original image to analyze their individual contribution to the final image color.

The Red, Green, and Blue channels are extracted as follows:

- **Red Channel:** contains only the red component; green and blue values are set to zero.
- **Green Channel:** contains only the green component; red and blue are zeroed.
- **Blue Channel:** contains only the blue component; red and green are set to zero.

Each extracted channel is displayed as an individual image. The red channel highlights reddish areas, the green channel emphasizes greenish regions, and the blue channel shows bluish tones of the original image.

This process is essential for color analysis, image enhancement, and applications like object recognition, segmentation, and color-based filtering

CODE:

```
% Extraction of Colour Band
```

```
clc;
```

```
clear all;
```

```
close all;
```

```
a = imread('lenna.jpeg');
```

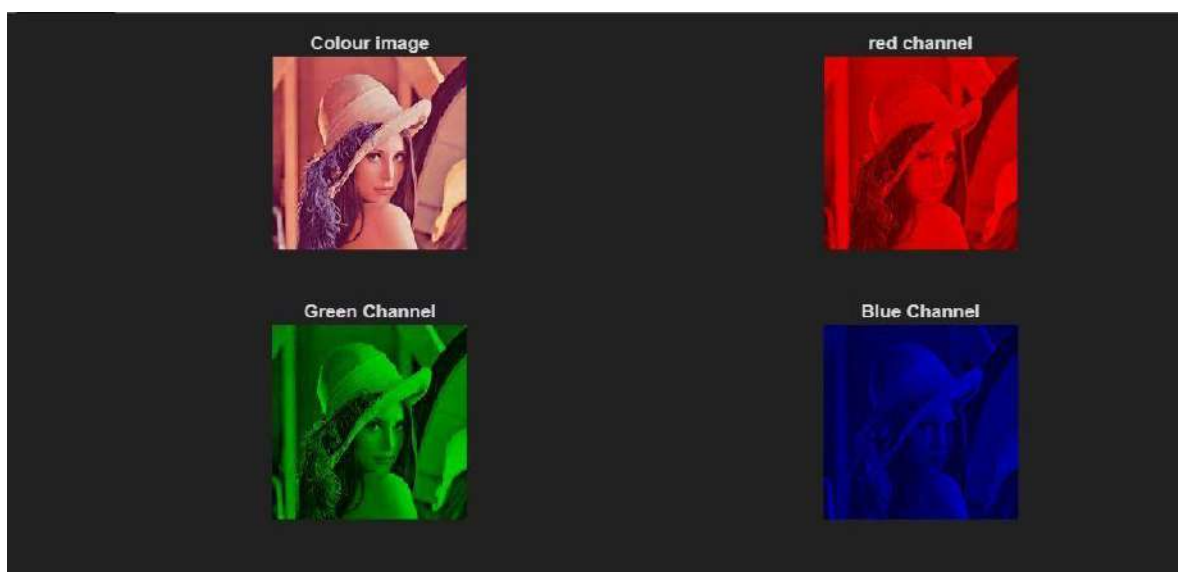
```
[r1 c1 chan]=size(a);
```

```
R=zeros(r1,c1,3);
```

```
G=zeros(r1,c1,3);
```

```
B=zeros(r1,c1,3);  
R(:,:,1)=a(:,:,1);  
G(:,:,2)=a(:,:,2);  
B(:,:,3)=a(:,:,3);  
subplot(2,2,1);  
imshow(a);  
title('Colour image');  
subplot(2,2,2);  
imshow(uint8(R));  
title('red channel');  
subplot(2,2,3);  
imshow(uint8(G));  
title('Green Channel');  
subplot(2,2,4);  
imshow(uint8(B));  
title('Blue Channel');
```

Result:



Experiment no: 4

AIM OF THE EXPERIMENT: Perform different Image resize.

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

An image in MATLAB is represented as a matrix of pixel intensity values. The size of the image is defined by the number of **rows** (height), **columns** (width), and **channels** (color components). When an image is resized, the total number of pixels changes accordingly.

The MATLAB function **imresize()** is used for image scaling and resizing. It uses interpolation methods such as **nearest-neighbor**, **bilinear**, or **bicubic interpolation** to estimate new pixel values after resizing.

In this experiment:

- The **original image** is first read and displayed.
- The image is **shrunk** by a scale factor of 0.5, reducing both rows and columns to half their size.
- The image is **expanded** by a factor of 2, doubling the number of rows and columns.
- The image is **resized to a specific dimension (100×100)** pixels, regardless of its original size.

Resizing is a fundamental operation in image processing, commonly used in applications like image compression, thumbnail generation, feature extraction, and machine learning preprocessing. It allows images to be adjusted for better analysis or for fitting within required display or storage dimensions

CODE:

%Image Resize

```
a=imread('Lenna_(test_image).png');
```

```
[r1,c1,chan]=size(a)
```

```
b = imresize(a,0.5);    %r -> r/2 , c -> c/2
```

```
c= imresize(a,2);      %r -> r*2 , c -> c*2
```

```
d=imresize(a,[100 100]); %r -> 100 , c -> 100
```

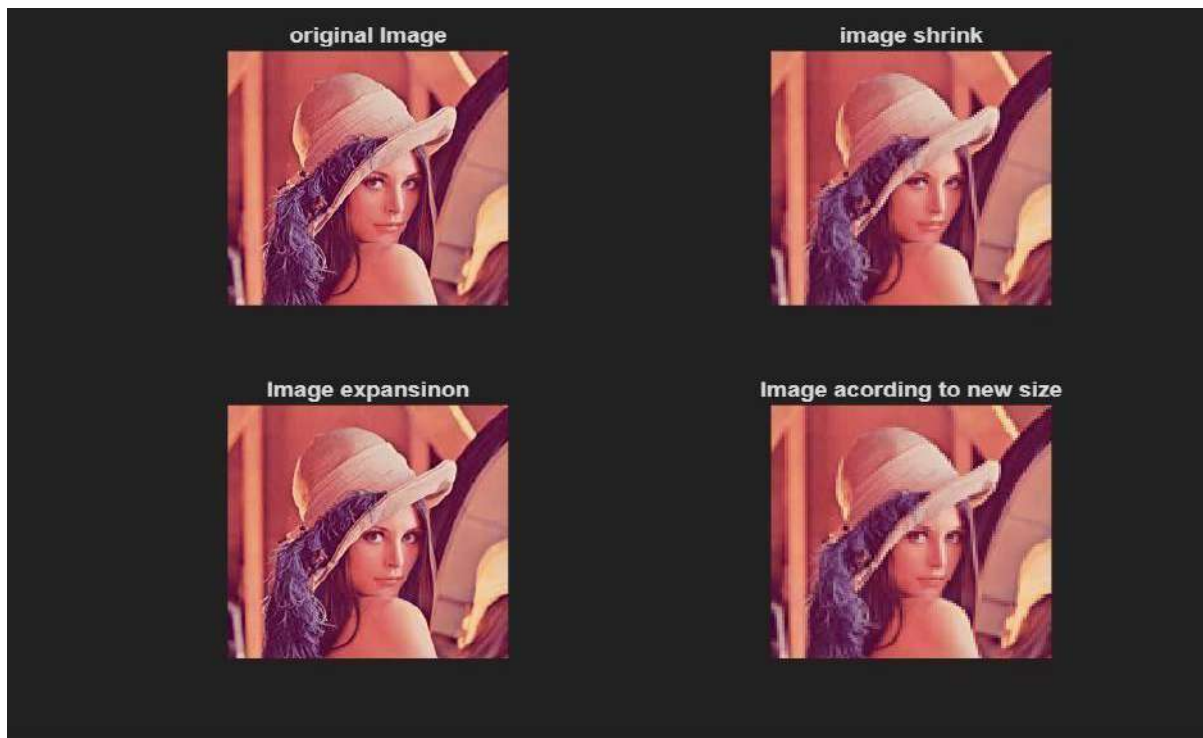
```
subplot(2,2,1);
```

```
imshow(a);
```

```
title('original Image');
```

```
subplot(2,2,2);  
imshow(b);  
title('image shrink');  
subplot(2,2,3);  
imshow(c);  
title('Image expansinon');  
subplot(2,2,4);  
imshow(d);  
title('Image acording to new size');
```

RESULT:



CONCLUSION:

In this experiment, image resizing operations were successfully performed using MATLAB. The original image was resized in different ways — reduced to half its size (shrink), enlarged to double its size (expansion), and adjusted to a fixed dimension (100×100 pixels).

The function **imresize()** effectively changed the number of rows and columns of the image matrix while maintaining the visual quality using interpolation techniques. This experiment demonstrates how resizing can be used to control image dimensions for various applications such as image compression, object recognition, and display adjustment.

Experiment no: 5

AIM OF THE EXPERIMENT: Perform different Image crop.

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

In MATLAB, the function **imcrop()** is used for cropping images. It extracts a rectangular portion from an image based on the coordinates and dimensions specified in the form [x y width height].

- **x, y** → specify the starting point (top-left corner) of the cropping region.
- **width, height** → specify the size of the region to be cropped.

In this experiment:

- The **original image** is first displayed.
- Different cropped portions of the image are extracted by changing the coordinate values and region size.
- Each cropped image shows a specific part of the original image, allowing analysis of selected regions.

Cropping is an essential operation in applications like image segmentation, object recognition, and region-based analysis, where only a certain portion of the image is required for further processing.

CODE:

```
%Image crop

clc;

clear all;

close all;

a= imread('Lenna_(test_image).png');

[r1,c1,chan]=size(a);

b=imcrop(a,[0 0 100 100]);

c=imcrop(a,[200 200 250 250]);

d=imcrop(a,[100 100 400 400]);

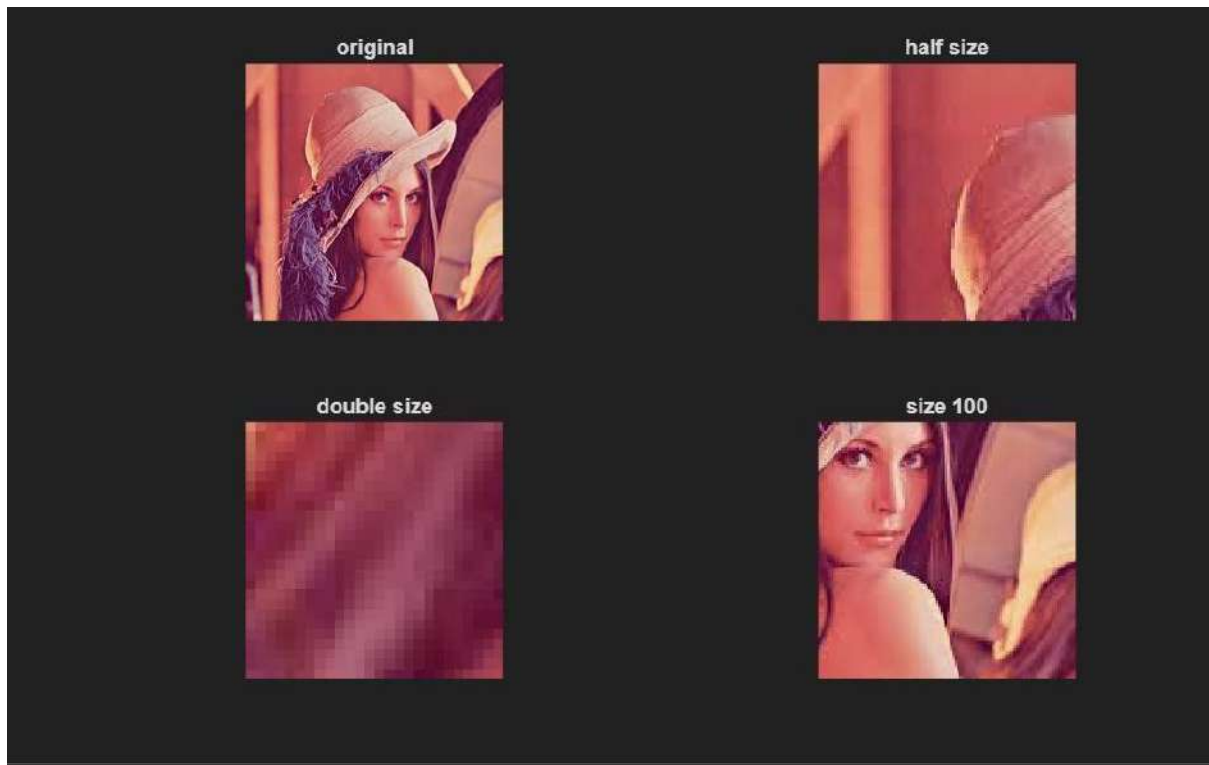
subplot(2,2,1);

imshow(a);
```



```
title('original');  
subplot(2,2,2);  
imshow(b);  
title('half size');  
subplot(2,2,3);  
imshow(c);  
title('double size');  
subplot(2,2,4);  
imshow(d);  
title('size 100');
```

RESULT:



Experiment no: 6

AIM OF THE EXPERIMENT: Perform different Image rotate.

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

In **Digital Image Processing**, **image rotation** is the process of rotating an image by a specific angle around a defined point, usually the center of the image. This operation changes the orientation of the image pixels without altering their intensity values.

In MATLAB, the function **imrotate()** is used to perform image rotation. It takes two main parameters — the image matrix and the rotation angle (in degrees).

- A **positive angle** rotates the image **counterclockwise**.
- A **negative angle** rotates the image **clockwise**.

In this experiment, the image is rotated by **45°**, **-45°**, and **90°** to observe how its orientation changes with each rotation. Rotation is widely used in applications such as image alignment, object tracking, geometric transformation, and pattern recognition.

CODE:

%Image Rotate

clc;

clear all;

close all;

a=imread('Lenna_(test_image).png');

[r1,c1,chan]=size(a);

b=imrotate(a,45);

c=imrotate(a,-45);

d=imrotate(a,90);

subplot(2,2,1);

imshow(a);

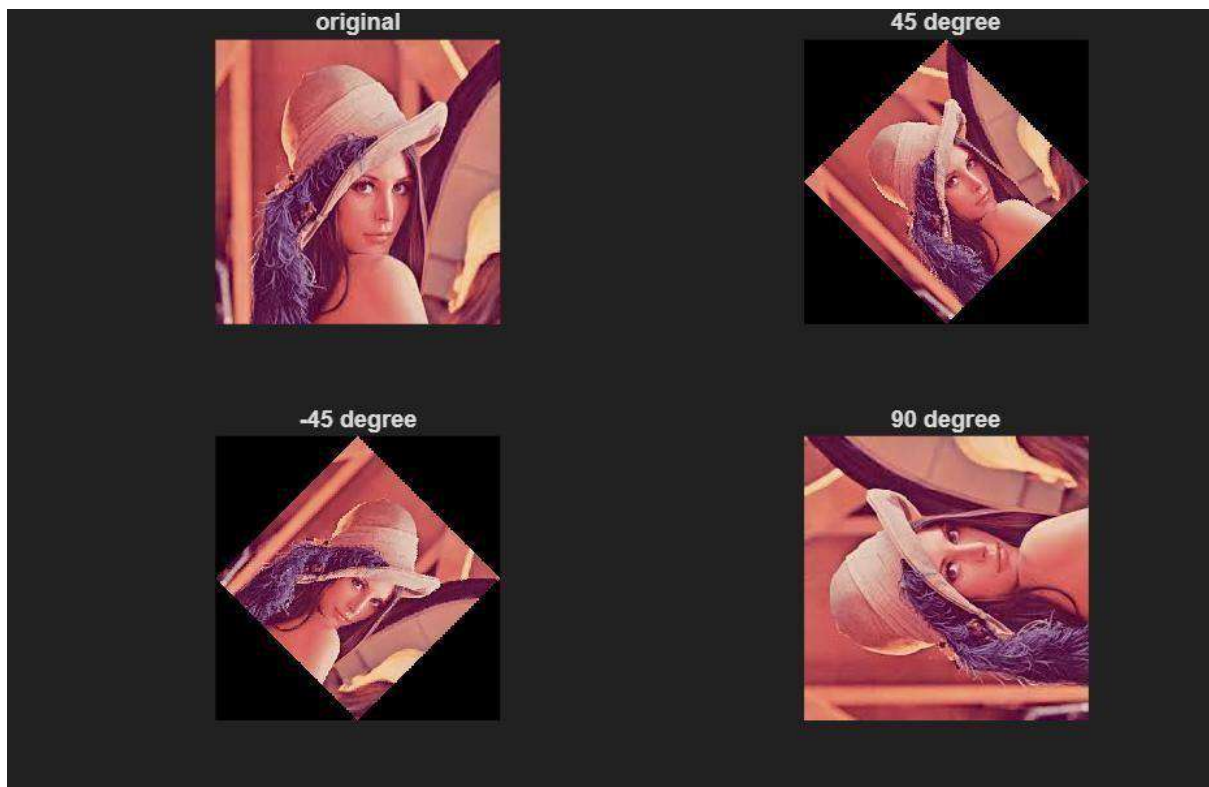
title('original');

subplot(2,2,2);

imshow(b);

```
title('45 degree');  
subplot(2,2,3);  
imshow(c);  
title('-45 degree');  
subplot(2,2,4);  
imshow(d);  
title('90 degree');
```

RESULT:



CONCLUSION:

From this experiment, we conclude that **image rotation** changes the orientation of an image while keeping its pixel intensity intact. The **imrotate()** function in MATLAB effectively rotates images by any desired angle, either clockwise or counterclockwise. This operation is essential in image alignment, geometric transformations, and computer vision applications.

Experiment no: 7

AIM OF THE EXPERIMENT: Write a program to plot different function like sy function, exponential wave and pdt of exponential and side by using stem command

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

This experiment demonstrates the plotting of **different mathematical functions** — sine, cosine, exponential, and exponential-sine — using **MATLAB's stem plots**.

- The **sine wave** is a periodic waveform that oscillates smoothly above and below zero, representing continuous wave signals used in communication and signal processing.
- The **cosine wave** is similar to the sine wave but phase-shifted by 90° , also used widely in analysing oscillatory systems.
- The **exponential curve** shows a rapid increase or decay in values depending on the exponent's sign; it represents growth and decay processes in real-world systems.
- The **exponential-sine curve** combines oscillatory and exponential behaviours, showing a sine wave whose amplitude changes exponentially over time.

These plots help visualize different signal behaviours in both the **time** and **amplitude** domains, essential for understanding the fundamentals of signal processing.

CODE:

```
a = 0 : pi/10 : 6*pi;
b = sin(a);
subplot(2,2,1);
stem(a,b,'r*--');
xlabel('time');
ylabel('amplitude');
title('sine wave');
c = 0 : pi/10 : 6*pi;
d = cos(c);
subplot(2,2,2);
stem(c,d,'bs-');
xlabel('time');
ylabel('amplitude');
```

```

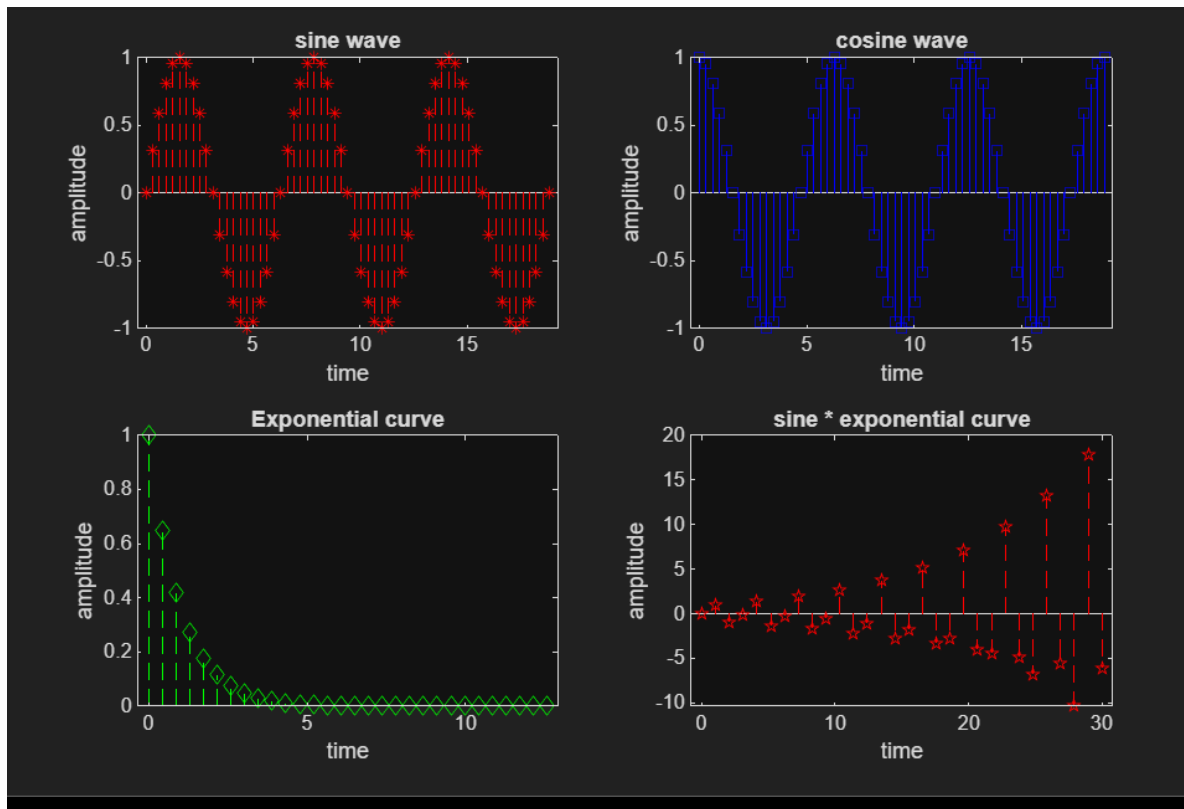
title('cos wave');

e=linspace(0,4*pi,30);
f=exp(-e);
subplot(2,2,3);
stem(e,f,'yd-');
xlabel('Amplitude');
ylabel('exponential curve');

g = linspace(0,30,30);
h = exp(g/10).*sin(2*g);
subplot(2,2,4);
stem(g,h,'go-');
xlabel('time');
ylabel('amplitude');
title('Sine * exponential curve');

```

RESULT:



Experiment no: 8

AIM OF THE EXPERIMENT: Perform various Image Enhancement operations: Image Negation function, Logarithmic Transformation, Power Law Transformation

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

This experiment demonstrates different **intensity transformation techniques** used in **Digital Image Processing** to enhance image features and modify brightness or contrast levels.

- **Image Negative:**

The negative transformation ($s = L - 1 - r$) inverts the intensity values of the image. Dark areas become light and vice versa. It is particularly useful for analyzing details in dark regions of an image, such as X-rays or satellite images.

- **Power Law (Gamma) Transformation:**

This technique applies the formula $s = c * r^\gamma$ to adjust image contrast and brightness. The value of γ (gamma) controls enhancement — $\gamma < 1$ brightens the image, while $\gamma > 1$ darkens it. This method is widely used in **display devices** and **image correction** applications.

- **Logarithmic Transformation:**

The log transformation enhances low-intensity pixels while compressing high-intensity values, following $s = c * \log(1 + r)$. It is useful for displaying images with large variations in pixel intensities, such as Fourier spectra or images with dark backgrounds.

Overall, these transformations help in **enhancing visual quality**, **highlighting features**, and **adjusting contrast** for better image interpretation.

CODE:

```
Img=imread("lenna.jpeg");  
Gray_Img=rgb2gray(Img);  
subplot(2,2,1);  
imshow(Gray_Img);  
%% image negative (L-1-r); L-1=maximum intensity r=intensity before  
% transformation ,s= after transformation  
L=256;  
Img_negative=L-1-Gray_Img;
```

```

subplot(2,2,2);
imshow(Img_negative); title("Image negative");

% Power law

C=1;
G=2;
r=double(Gray_Img);
S=C*(r.^G);
T=255/(C*(255.^G));
% Converting the datatype back to image integer for displaying
O = uint8(T * S);
subplot(2,2,3);
imshow(O),title("Power law tranformed image")

```

% log transformation

```

L=C*log(1+r);
S=C*(r.^G);
T=255/(C*log(256));
% Converting the datatype back to image integer for displaying
Log_Img = uint8(T * L);
subplot(2,2,4);
imshow(Log_Img),title(" log tranformed image")

```

Conclusion :

In this experiment, various intensity transformation techniques such as Image Negative, Power Law (Gamma), and Logarithmic Transformation were successfully implemented using MATLAB. Each transformation altered the pixel intensity distribution to enhance image visibility and contrast.

The **negative transformation** highlighted dark regions, **power law** adjusted image brightness and contrast, and **log transformation** enhanced low-intensity details. Overall, these operations are essential for **image enhancement**, **visual analysis**, and **feature extraction** in digital image processing.

RESULT:



Image negative



Power law tranformed image



log tranformed image



Experiment no: 9

AIM OF THE EXPERIMENT: Perform various Image Enhancement operations: Histogram Equalization, contrast stretching, plot histogram without using imhist function

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

This experiment demonstrates **Histogram Equalization**, a powerful image enhancement technique used in **Digital Image Processing** to improve image contrast.

A **histogram** represents the distribution of pixel intensity values in an image. In many images, these intensities are concentrated within a narrow range, resulting in low contrast and poor visibility of details.

Histogram Equalization redistributes the pixel intensity values so that they span the entire available range (0–255 for 8-bit images). This process enhances contrast by making dark areas lighter and bright areas darker in a balanced way.

The function **histeq()** in MATLAB performs this operation automatically by computing the cumulative distribution function (CDF) of intensity levels and mapping them to new values. As a result, the output image appears clearer, with improved brightness and contrast, making fine details more visible and suitable for further analysis.

CODE:

```
% Histogram Equalization

clc;

clear all;

close all;

a=imread('DIP5.tif');

b=histeq(a);

subplot(2,2,1);

imshow(a);

title('input image');

subplot(2,2,2);

imhist(a);

title('Histogram of input image');

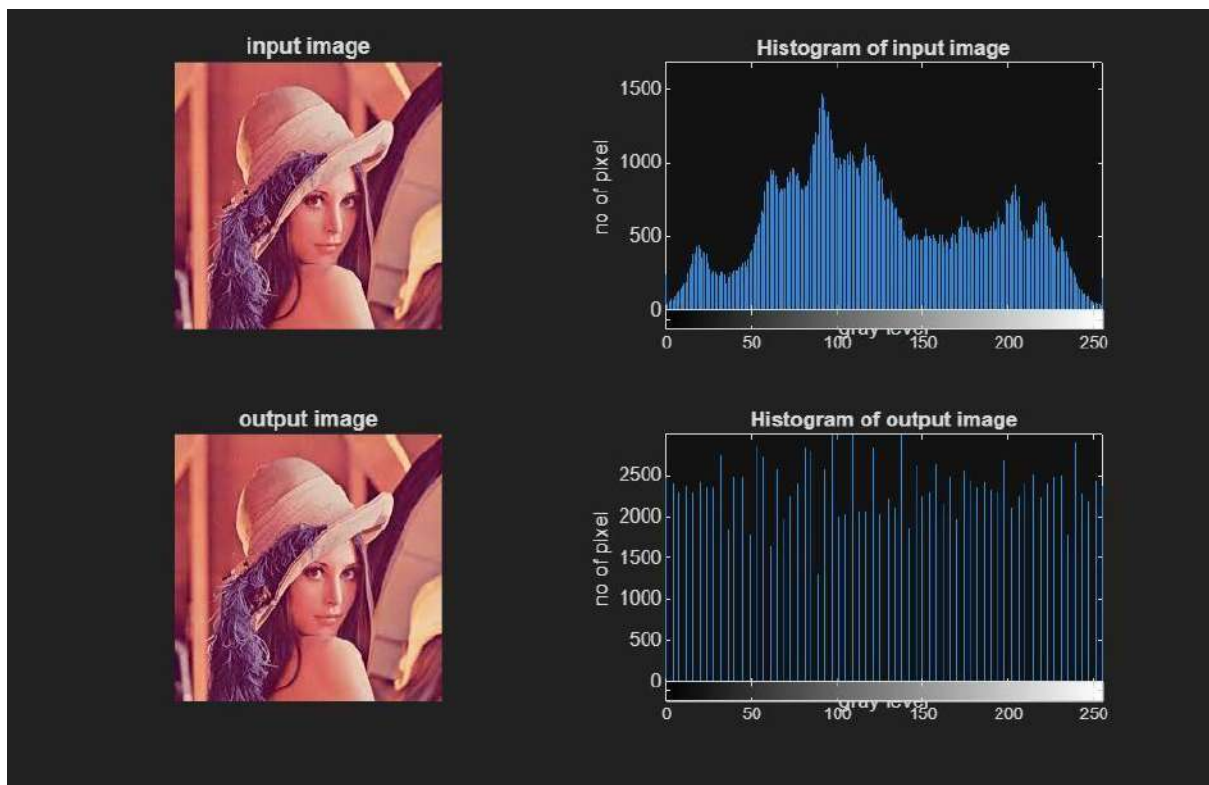
xlabel('gray level');
```

```

ylabel('no of pixel');
subplot(2,2,3);
imshow(a);
title('output image');
subplot(2,2,4);
imhist(b);
title('Histogram of output image');
xlabel('gray level');
ylabel('no of pixel');

```

RESULT:



Conclusion:

In this experiment, **Histogram Equalization** was successfully applied to enhance the contrast of an image. The technique redistributed the intensity values across the full range, resulting in improved brightness and visibility of details. The output image showed a more uniform histogram, proving that this method effectively enhances low-contrast images and is highly useful in applications like medical imaging and remote sensing.

Experiment no: 10

AIM OF THE EXPERIMENT: Sharpen an image using Laplacian filter.

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

This experiment demonstrates **edge detection using the Laplacian operator** in **Digital Image Processing**. Edge detection is a fundamental technique used to identify boundaries and sharp intensity changes in an image, which represent important structural features.

The **Laplacian operator** is a **second-order derivative filter** that highlights regions of rapid intensity variation by computing the rate of change in all directions. In this code:

- The image is first converted to **grayscale** to simplify processing since color information is not needed for edge detection.
- Two Laplacian masks are used:
 - **L4 (4-connected Laplacian):** Detects edges considering only the four immediate neighbors (top, bottom, left, right).
 - **L8 (8-connected Laplacian):** Considers all eight surrounding pixels, providing stronger edge responses.
- The **imfilter()** function performs convolution of the grayscale image with these masks to extract edge information.
- The difference between the original image and the Laplacian-filtered image enhances **fine details and textures**.

Overall, the Laplacian method helps in identifying and enhancing edges, making it useful for applications like object recognition, segmentation, and image sharpening.

CODE:

```
clc;
```

```
clear all;
```

```
close all;
```

```
Img = imread('lenna.jpeg');
```

```
Gray_Img = rgb2gray(Img);
```

```
subplot(2,3,1);
```

```
imshow(Gray_Img);
```

```
title('Original Image');
```

```
%noisy_img = imnoise(Gray_Img, 'salt & pepper');
```

```
%subplot(2,3,2);
```

```
%imshow(noisy_img);
```

```
%title('Image with Salt & Pepper Noise');
```

```
L4 = [0 -1 0; -1 4 -1; 0 -1 0];
```

```
L8 = [1 1 1; 1 -8 1; 1 1 1];
```

```
conv_L4 = imfilter(double(Gray_Img), L4, 'replicate');
```

```
conv_L8 = imfilter(double(Gray_Img), L8, 'replicate');
```

```
edges_L4 = uint8(conv_L4);
```

```
edges_L8 = uint8(conv_L8);
```

```
details_L4 = (Gray_Img) - uint8(conv_L4);
```

```
details_L8 = (Gray_Img) - uint8(conv_L8);
```

```
subplot(2,3,2);
```

```
imshow(edges_L4, []);
```

```
title('Edges using L4 Laplacian');
```

```
subplot(2,3,3);
```

```
imshow(edges_L8, []);
```

```
title('Edges using L8 Laplacian');
```

```
subplot(2,3,4);
```

```
imshow(uint8(details_L4));
```

```
title('Details (L4 Laplacian)');
```

```
subplot(2,3,5);
```

```
imshow(uint8(details_L8));
```

```
title('Details (L8 Laplacian)');
```

RESULT:



Conclusion:

In this experiment, **edge detection using the Laplacian operator** was successfully performed. Both **L4** and **L8 Laplacian masks** were applied to the grayscale image to extract edges and enhance fine details. The results showed that the L8 mask provided stronger and more detailed edges compared to L4. This technique effectively highlights intensity transitions and is widely used for **image sharpening, feature extraction, and object boundary detection** in image processing.

Experiment no: 11

AIM OF THE EXPERIMENT: Perform various Fast Fourier transform (FFT) and frequency domain filtering on images using MATLAB.

SOFTWARE REQUIRED: MATLAB MATH-WORKS

THEORY:

Fast Fourier Transform (FFT) and Frequency Domain Filtering

In **Digital Image Processing**, the **Fast Fourier Transform (FFT)** is a powerful mathematical technique used to convert an image from the **spatial domain** (where pixels represent intensity) into the **frequency domain** (where pixels represent frequency components). This transformation helps in analyzing and modifying the image based on its frequency content.

◆ **Fast Fourier Transform (FFT):**

- The **FFT** is an efficient algorithm for computing the **Discrete Fourier Transform (DFT)** of an image.
- It decomposes an image into **sinusoidal components** of varying frequencies and amplitudes.
- The resulting frequency-domain image contains:
 - **Low-frequency components:** Represent smooth regions and background information.
 - **High-frequency components:** Represent edges, noise, and fine details.
- In MATLAB, the function **fft2()** is used to compute the 2D Fourier transform, and **ifft2()** performs the inverse transform to get back the spatial image.
- The **fftshift()** function centers the low-frequency components in the output for better visualization.

◆ **Frequency Domain Filtering:**

Frequency domain filtering involves modifying the **Fourier-transformed image** to enhance or suppress certain frequency components before converting it back to the spatial domain.

Common types of filters include:

1. **Low Pass Filter (LPF):**
 - Allows low frequencies (smooth areas) to pass and blocks high frequencies (edges/noise).
 - Used for **image smoothing** and **noise reduction**.
2. **High Pass Filter (HPF):**

- Allows high frequencies to pass and suppresses low frequencies.
- Used for **edge detection** and **image sharpening**.

3. Band Pass / Band Reject Filters:

- Band Pass allows only a specific range of frequencies.
- Band Reject removes a specific frequency range (used in periodic noise removal).

◆ Applications:

- Image enhancement and restoration
- Edge and feature extraction
- Noise reduction
- Image compression and reconstruction

CODE:

```
%load image
```

```
f=imread('lenna.jpeg');
```

```
[M,N] = size (f);
```

```
P=2*M;
```

```
Q=2*N;
```

```
%FFt and shifting
```

```
F= fft2(f,P,Q);
```

```
Fc= fftshift(F);
```

```
S2= log(1+abs(Fc));
```

```
%create ideal low pass filter
```

```
H=zeros(P,Q);
```

```
DO=input('Enter the value DO= '); %cutoff frequency
```

```
d = zeros(P,Q);
```

```
for i=1:P
```

```
    for j=1:Q
```

```
        d(i,j)=sqrt((i-P/2)^2 + (j - Q/2)^2;
```

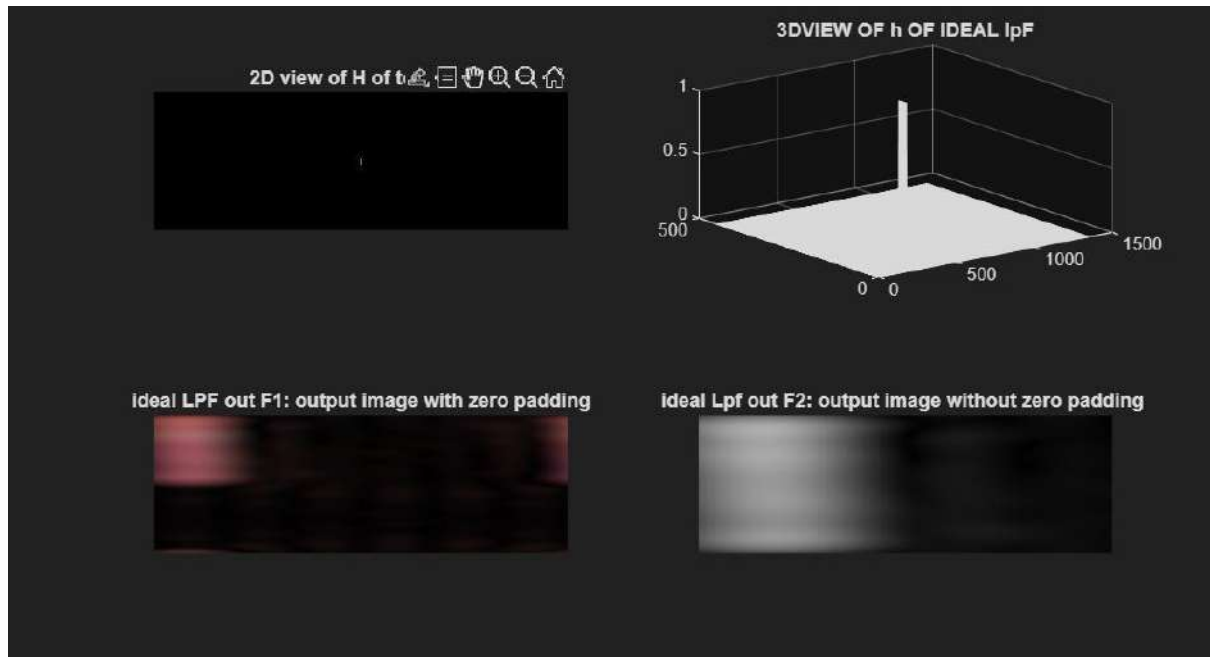
```

        H(i,j) = (d(i,j) <= DO);
    end
end
%inverse transform
out = H .* Fc;
outC=ifftshift(out);
outF1=ifft2(outC);
outF2=outF1(1:M, 1:N);%crop to original size

figure ; subplot(2,2,1);
imshow(f) ; title ('original image f');
subplot(2,2,2);
imshow(abs(F), []); title('fourier spectrum F');
subplot(2,2,3);
imshow(abs(Fc), []); title('centered fourier specturm Fc');
subplot(2,2,4);
imshow(S2,[]); title('S2: log transformation Fc');

figure;
subplot(2,2,1);
imshow(H, []); title('2D view of H of total Lpf');
subplot(2,2,2); surf(H); title('3DVIEW OF h OF IDEAL lpF');
subplot(2,2,3); imshow(uint8(abs(outF1))); title('ideal LPF out F1: output image with zero padding');
subplot(2,2,4); imshow(uint8(abs(outF2))); title('ideal Lpf out F2: output image without zero padding
');
```


RESULT:



CONCLUSION:

In this experiment, the **Fast Fourier Transform (FFT)** and **frequency domain filtering** techniques were successfully studied and implemented. The FFT efficiently converted the image from the spatial domain to the frequency domain, allowing analysis of its low- and high-frequency components. By applying **Low Pass** and **High Pass filters**, image details and noise were effectively controlled. This approach proved highly useful for **image enhancement, noise removal, and feature extraction**, making FFT an essential tool in digital image processing.